

Contents

MuTau Channel: Proton Mixing & Weight Normalization	2
Analysis Status Report	2
1. Analysis Overview	2
Goal	2
Samples Used	2
2. Processing Pipeline	2
3. Phase 0: Initial Skimming	3
Scripts	3
What it does	3
Proton Variables Saved (Data only)	3
Output	3
4. Phase 1: Physics Selection	3
Scripts	3
Selection Cuts Applied	4
5. Event Weights in Phase 1	4
Data	4
Drell-Yan (DY)	4
ttbar	4
QCD	4
6. The Proton Pool	4
What is it?	4
Location	4
Contents	5
How it was created	5
Why it works	5
7. Proton Mixing Algorithm	5
Purpose	5
Script	5
Step 1: Always assign 1 proton per arm	5
Step 2: Probabilistically add 2nd protons	5
Output Branches	6
8. The Proton Acceptance Factor	6
What is it?	6
Why is it needed?	6
Measurement from Data	6
Breakdown	6
Historical Value	6
Discrepancy	6
9. The Bug We Found	7
Original Code (WRONG)	7
Fixed Code (CORRECT)	7
10. Complete Weight Flow	7
Data (no proton mixing needed)	7
Drell-Yan	7
ttbar	8
QCD (data-driven)	8

11. Cross-Section Normalization Factors	8
Formula	8
Values Used	8
12. Current Status of Plotting Code	8
File: <code>plot_m.py</code>	8
How weights are currently used:	8
Issues:	9
13. Summary of Issues Found	9
14. Next Steps	9
Immediate	9
Verification	9
15. Files Summary	9
Processing Scripts	9
Output Samples	10
16. Proton Acceptance Measurement Code	10
Quick method in ROOT:	10
Result from my measurement:	10
Questions for Discussion	10

MuTau Channel: Proton Mixing & Weight Normalization

Analysis Status Report

1. Analysis Overview

Goal

Search for exclusive di-tau production in the muon-tau channel using 2018 data with the Precision Proton Spectrometer (PPS).

Samples Used

Sample	Description	Source
Data	2018 UL SingleMuon	Real collision data
DY	Drell-Yan $Z/\gamma^* \rightarrow$	MC simulation
ttbar	$tt \rightarrow$ + X	MC simulation
QCD	Multi-jet background	Data-driven (same-sign)
Signal	Exclusive $\rightarrow$	MC simulation

2. Processing Pipeline

NanoAOD (CMS)	Phase 0 Skimming	Phase 1 Selection	Merge Files
------------------	---------------------	----------------------	----------------

Plots	Proton Mixing	Merged Phase 1 Files (DY, ttbar, Data, QCD)
-------	------------------	--

3. Phase 0: Initial Skimming

Scripts

- `fase0_data_mutau.py` (Data)
- `fase0_dy.py` (Drell-Yan)

What it does

1. **Trigger Selection:** HLT_IsoMu24 (single muon trigger for 2018)
2. **Luminosity Filtering:** Only certified good runs (golden JSON)
3. **Basic Object Selection:** At least 1 muon and 1 tau
4. **Variable Extraction:**
 - Muon: pT, η , ϕ , mass, charge, ID
 - Tau: pT, η , ϕ , mass, charge, DeepTau IDs
 - Jets: pT, η , ϕ , mass, b-tag
 - MET: pT,

Proton Variables Saved (Data only)

```
nproton_multi, nproton_single
proton_multi_xi, proton_multi_arm, proton_multi_t
proton_multi_thetaX, proton_multi_thetaY
proton_multi_time, proton_multi_timeUnc
proton_single_xi
```

Output

- Skimmed ROOT files with `tree` containing selected events
 - `weight = 1.0, generator_weight = 1.0` (for Data)
-

4. Phase 1: Physics Selection

Scripts

- `fase1_data.py` (Data)
- `fase1_dy.py` (Drell-Yan)
- `fase1_ttjets.py` (ttbar)
- `fase1_qcd.py` (QCD)

Selection Cuts Applied

Variable	Cut	Description
muon_id	> 3	Medium MVA ID
tau_id1 (VSjet)	> 63	VeryTight DeepTau vs jets
tau_id2 (VSe)	> 7	Tight DeepTau vs electrons
tau_id3 (VSmu)	> 1	Loose DeepTau vs muons
muon_pt	> 35 GeV	Muon transverse momentum
tau_pt	> 100 GeV	Tau transverse momentum
muon_charge × tau_charge	< 0	Opposite sign (OS)
$\Delta R(,)$	> 0.4	Angular separation
	< 2.4	Detector acceptance

Note: QCD uses **same-sign** (SS) selection for data-driven estimate.

5. Event Weights in Phase 1

Data

```
event_weight = 1.0 # No correction needed
```

Drell-Yan (DY)

```
event_weight = generator_weight × mu_trig_sf × mu_idiso_sf × mu_reco_sf
```

Where: - **generator_weight**: MC generator weight (NLO corrections, etc.) - **mu_trig_sf**: Muon trigger scale factor from POG - **mu_idiso_sf**: Muon ID and isolation scale factor - **mu_reco_sf**: Muon reconstruction scale factor

ttbar

```
event_weight = 0.15 × mu_trig_sf × mu_idiso_sf × mu_reco_sf
```

Important: The cross-section normalization factor (0.15) is **already included** at this stage!

QCD

```
event_weight = 1.0 # Data-driven, normalized separately
```

6. The Proton Pool

What is it?

A collection of real protons from 2018 PPS data, used to simulate pileup protons in MC.

Location

```
/eos/cms/store/group/phys_smp/Exclusive_DiTau/proton_pool_2018/proton_pool_2018.root
```

Contents

Branch	Description
proton_arm	Detector arm (0 = +z, 1 = -z)
proton_xi	Fractional momentum loss = $(p_{\text{beam}} - p_{\text{proton}})/p_{\text{beam}}$

How it was created

1. Take 2018 data with signal triggers
2. Apply **no kinematic cuts** on the central system
3. Extract all detected protons from PPS
4. These are essentially random pileup protons

Why it works

- Protons have correct distribution for 2018 running conditions
 - Includes proper detector acceptance and efficiency effects
 - Random sampling gives realistic pileup simulation
-

7. Proton Mixing Algorithm

Purpose

MC samples (DY, ttbar) don't have PPS proton information. We need to add realistic pileup protons to compare with data.

Script

`merge_pp_mutau.py`

Step 1: Always assign 1 proton per arm

```
# Loop through proton pool until we find:  
# - 1 proton with arm=0 → store as xi_arm1_1  
# - 1 proton with arm=1 → store as xi_arm2_1
```

Step 2: Probabilistically add 2nd protons

Configuration	Probability	Description
P	8.0%	1 proton on arm0, 1 proton on arm1
P	2.0%	1 proton on arm0, 2 protons on arm1
P	2.0%	2 protons on arm0, 1 proton on arm1
P	0.5%	2 protons on each arm

Output Branches

Branch	Description
xi_arm1_1	1st proton on arm 0 (always filled)
xi_arm1_2	2nd proton on arm 0 (-1 if none)
xi_arm2_1	1st proton on arm 1 (always filled)
xi_arm2_2	2nd proton on arm 1 (-1 if none)

8. The Proton Acceptance Factor

What is it?

The probability that a random event has **at least 1 proton detected on BOTH arms** simultaneously.

Why is it needed?

- After proton mixing, **100% of MC events** have 1 proton per arm
- In real data, only a **fraction** of events have this topology
- We must weight MC to account for this

Measurement from Data

Sample: Data_2018_UL_MuTau_nano_merged_proton_vars.root

Total events: 6,970

Events with 1 proton on both arms: 1,708

Proton acceptance $P = 24.5\%$

Breakdown

Category	Events	Fraction
No protons on either arm	1,849	26.5%
Protons only on arm 0	1,600	23.0%
Protons only on arm 1	1,813	26.0%
Protons on BOTH arms	1,708	24.5%

Historical Value

Matteo's original measurement: **$P = 0.13$ (13%)**

Discrepancy

The difference (13% vs 24.5%) may be due to: - Different event selection stage - Different data sample or time period - Different trigger requirements

Action needed: Clarify which value is correct for this analysis.

9. The Bug We Found

Original Code (WRONG)

```
# In merge_pp_mutau.py, line 143-144:
weight = array("d", [float(FIXED_WEIGHT)])
t_out.Branch("weight", weight, "weight/D")
```

```
# In event loop:
# weight was never updated - always 0.13!
```

Problem: The original event_weight (containing generator weights and scale factors) was **completely discarded**.

Fixed Code (CORRECT)

```
# Read input event_weight if it exists
has_event_weight = ensure_branch(t_in, "event_weight")
if has_event_weight:
    event_weight_in = array("d", [1.0])
    t_in.SetBranchAddress("event_weight", event_weight_in)

# In event loop:
if event_weight_in is not None:
    weight[0] = event_weight_in[0] * FIXED_WEIGHT
else:
    weight[0] = FIXED_WEIGHT
```

Result: Now $\text{weight} = \text{event_weight} \times 0.13$ (preserves all corrections).

10. Complete Weight Flow

Data (no proton mixing needed)

```
Phase 0: weight = 1.0
Phase 1: weight = 1.0
Plotting: final_weight = 1.0
```

Drell-Yan

```
Phase 1:
    event_weight = generator_weight * muon_SF
```

```
Proton Mixing:
    weight = event_weight * 0.13
```

```
Plotting:
    final_weight = weight * 1.81 (cross-section normalization)
```

ttbar

Phase 1:

event_weight = 0.15 × muon_SF's ← 0.15 already included!

Proton Mixing:

weight = event_weight × 0.13

Plotting:

final_weight = weight × 1.0 ← no additional factor needed

QCD (data-driven)

Phase 1: weight = 1.0 (same-sign selection)

Plotting: final_weight = custom normalization

11. Cross-Section Normalization Factors

Formula

weight = N_{MC} / N_{expected}

where N_{expected} = Luminosity × Cross-section

Values Used

Sample	Factor	Included Where?
DY	1.81	Applied in plotting
ttbar	0.15	Already in Phase 1 event_weight

12. Current Status of Plotting Code

File: plot_m.py

How weights are currently used:

```
# QCD: fixed weight
```

```
w_qcd = 2.0
```

```
# DY: reads weight from tree (currently contains only 0.13)
```

```
w_dy = get_value(event, "weight", default=1.81)
```

```
# ttbar: reads weight from tree (currently contains only 0.13)
```

```
w_ttjets = get_value(event, "weight", default=0.15)
```


Issues:

1. DY and ttbar files have `weight = 0.13` (not including `event_weight`)
 2. DY needs additional 1.81 factor not being applied
 3. ttbar has 0.15 in `event_weight` but it's lost in current merged files
-

13. Summary of Issues Found

Issue	Status	Action Required
Proton mixing overwrites weights	Fixed	Re-run proton mixing
DY cross-section factor missing	Pending	Apply 1.81 in plotting
Proton acceptance value unclear	Pending	Decide: 0.13 vs 0.245
ttbar <code>event_weight</code> lost	Fixed	Re-run proton mixing

14. Next Steps

Immediate

1. **Decide on proton acceptance value** (discuss with Jonathan/Matteo)
2. **Re-run proton mixing** for DY and ttbar with fixed script
3. **Update plotting code** to apply correct normalizations

Verification

4. **Generate comparison plots** before/after fix
 5. **Cross-check with Jonathan's results**
-

15. Files Summary

Processing Scripts

Script	Purpose
<code>fase0_data_mutau.py</code>	Phase 0 for Data
<code>fase0_dy.py</code>	Phase 0 for DY
<code>fase1_data.py</code>	Phase 1 for Data
<code>fase1_dy.py</code>	Phase 1 for DY
<code>fase1_ttjets.py</code>	Phase 1 for ttbar
<code>fase1_qcd.py</code>	Phase 1 for QCD
<code>merge_pp_mutau.py</code>	Proton mixing

Script	Purpose
plot_m.py	Plotting

Output Samples

File	Description
Data_2018_UL_MuTau_nano_merged_data_proton_vars.root	Data with one proton
DY_2018_UL_MuTau_nano_merged_DY_with_protons.root	DY with protons
ttjets_2018_UL_MuTau_nano_merged_with_two_protons.root	tt with two protons
QCD_2018_UL_MuTau_nano_merged_QCD_data_vars.root	QCD (data vars)

16. Proton Acceptance Measurement Code

Quick method in ROOT:

```
// Open file
TFile* f = TFile::Open("Data_2018_UL_MuTau_nano_merged_proton_vars.root");
TTree* tree = (TTree*)f->Get("tree");

// Count total events
Long64_t total = tree->GetEntries();

// Count events with 1 proton on both arms
Long64_t good = tree->GetEntries(
    "Sum$(proton_multi_arm==0)>=1 && Sum$(proton_multi_arm==1)>=1"
);

// Calculate acceptance
double P = (double)good / (double)total;
cout << "Proton acceptance P = " << P << endl;
```

Result from my measurement:

Total events: 6,970
 Good events: 1,708
 P = 0.245 (24.5%)

Questions for Discussion

1. Which proton acceptance value should we use?
 - Matteo's 0.13 or my measured 0.245?
 - What selection was used for Matteo's measurement?
2. Is the cross-section normalization correct?
 - DY: 1.81

- ttbar: 0.15 (already in event_weight)
3. **Should we re-process all samples?**
- Or just re-run proton mixing with the fix?

Prepared for MuTau Analysis Meeting Date: January 2025